

05 · Inter-Task-Kommunikation und Interrupts

Queues, Mailboxes und ISR-Anbindung · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 45

Willkommen zurück zu Echtzeitsysteme!

- Herzlich willkommen zum 5. Termin unserer Vorlesung
- Halbzeit! Sie kennen nun die Grundlagen von Scheduling und Ressourcenverwaltung
- Letzte Woche: Die dunkle Seite der Präemption (Race Conditions, Priority Inversion)
- Heute: Wie transportieren wir Daten sicher von A nach B?
- Außerdem: Wie binden wir Hardware-Interrupts professionell an ein RTOS an?

Notizen

Guten Morgen und herzlich willkommen zur fünften Vorlesung! Wir haben Bergfest, die Hälfte des Semesters ist geschafft. Sie haben bis hierhin ein tiefes Verständnis dafür entwickelt, wie der Scheduler funktioniert und welche Gefahren lauern, wenn wir Variablen einfach wild zwischen Tasks teilen. Letzte Woche: Mutexe zum Schutz geteilter Daten. Heute: Daten aktiv und sicher transportieren. Im zweiten Teil: Hardware-Interrupts fehlerfrei in FreeRTOS integrieren.

Folie

2 / 45

Agenda für heute

1. Recap: Mutexe und Priority Inversion
2. Das Problem: Transport kontinuierlicher Datenströme
3. Inter-Task Communication (ITC): Queues
4. CMSIS-API für Queues und Value vs. Reference
5. Der Spezialfall: Mailboxes (Status vs. Events)
6. *[Nach der Pause]* Interrupts im RTOS: Top-Half und Bottom-Half
7. Ausblick Labor: Deferred Interrupt Processing am STM32H755

Notizen

Zuerst: Warum ein Mutex für Datenströme ungeeignet ist. Dann Inter-Task Communication (ITC) und Warteschlangen (Queues) — FreeRTOS kennt Tasks, keine Prozesse. C-API, Pointer vs. Kopie, Mailbox.

Folie

3 / 45

Recap: Die Lehren aus Termin 4

- Race Conditions: asynchrone Präemption in geteilten Variablen (Read-Modify-Write zerreißt)
- Mutex: exklusives Schloss mit Besitzrecht (Ownership) für Critical Sections
- Priority Inversion: Low-Task blockiert High-Task
- Rettung (Priority Inheritance): OS boostet die Low-Task temporär auf die Priorität der wartenden High-Task

Notizen

Kurzer Rückblick: `counter++` sind drei Assembler-Befehle — SysTick dazwischen → Race Condition. Mutexe schützen. Prioritätsinversion (Pathfinder). CMSIS-Mutex löst das über Priority Inheritance in FreeRTOS.

Folie

4 / 45

Mutexe schützen, aber transportieren nicht!

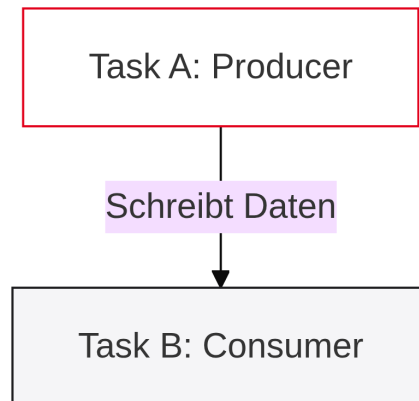
- Ein Mutex eignet sich perfekt für:
 - Den UART-Port (eine Task schreibt, die andere wartet)
 - Ein Setup-Struct (einmal schreiben, oft lesen)
- Problem: kontinuierlicher Datenstrom (Audio-Samples, Sensordaten)
 - Task A produziert jede Millisekunde einen neuen Wert
 - Task B liest und filtert
 - Globale Variable + Mutex → mehr Zeit am Schloss als beim Rechnen

Notizen

Mutexe sind Zugangskontrolle, nicht Transport. Bei 44.000 Samples/s wäre jedes Sample sperren/entsperren ein Flaschenhals: CPU rechnet Mutex-Logik statt Audiofilter. Wir brauchen einen besseren Weg.

Motivation: Producer & Consumer

- Das häufigste Software-Muster in eingebetteten Systemen
- Producer (Erzeuger): liest Sensoren, empfängt Pakete, generiert schnell Rohdaten
- Consumer (Verbraucher): filtert, rechnet, Display — oft langsamer oder in Blöcken
- Leitfrage: Wie überbrücken wir unterschiedliche Geschwindigkeiten sicher?



Notizen

Producer-Consumer-Pattern: schneller Erzeuger (z. B. ADC) und langsamerer Verbraucher (FFT, Display). Geschwindigkeitsgefälle → zeitliche Entkopplung nötig.

Die Falle der globalen Variable

- Annahme: globale Variable `adc_value`
- Producer überschreibt jede 1 ms
- Consumer kommt nur alle 5 ms dazu

```
// Producer (jede 1 ms)
void Task_ADC() {
    adc_value = Hardware_Read();
}

// Consumer (alle 5 ms)
void Task_Filter() {
    // Nur EIN Wert da – die 4 vorherigen sind überschrieben!
}
```

- Ergebnis: Datenverlust, obwohl das OS fehlerfrei läuft

Notizen

Producer feuert jede ms, Consumer alle 5 ms (niedrige Prio oder FFT). Consumer sieht nur den letzten Wert; vier Werte davor sind verloren — massiver Datenverlust bei korrektem OS.

Folie

7 / 45

Die Lösung: Inter-Task Communication (ITC)

- Wir brauchen Inter-Task Communication (ITC) — in FreeRTOS: Task-zu-Task, keine Prozesse
- (In der Literatur oft „IPC“ genannt; hier meinen wir dasselbe Muster auf Task-Ebene.)
- Das Betriebssystem muss die Datenverwaltung übernehmen
- Lösung: ein Puffer (Buffer)
 - Speicherbereich für mehrere Werte
 - Producer legt Daten vorne hinein
 - Consumer holt Daten hinten heraus
- In RTOS-Systemen: Queue (Warteschlange)

Notizen

Puffer entkoppelt Geschwindigkeiten. Producer legt ab, OS stapelt, Consumer holt in Ruhe. In FreeRTOS heißt dieser Puffer Queue — Rückgrat der Inter-Task-Kommunikation.

Folie

8 / 45

Lernziele des heutigen Tages

1. Das Producer-Consumer-Pattern verstehen
2. Queues in CMSIS-RTOS anlegen, beschreiben und lesen
3. Vor- und Nachteile von Copy by Value vs. Copy by Reference kennen
4. Semantischen Unterschied Queue vs. Mailbox erklären
5. *[Nach der Pause]* Hardware-Interrupts über Deferred Interrupt Processing anbinden

Notizen

Block 1: Queues, API, Value vs. Reference, Mailbox. Nach der Pause: Deferred Interrupt Processing — ISR-Latenz minimieren.

Die Queue (Warteschlange)

- Queue = Array im RAM, exklusiv vom OS verwaltet
- FIFO-Prinzip (First-In, First-Out)
 - Was zuerst hineinlegt wird, kommt zuerst heraus (Supermarktkasse)
- Threadsicher: FreeRTOS schützt intern mit Interrupt-Sperren
- Keine eigenen Mutexe nötig beim Schreiben in eine Queue

Notizen

Technisch ein Array, verwaltet von FreeRTOS. FIFO erhält die Reihenfolge. Thread-safe: kein `osMutexAcquire` vor dem Schreiben — das OS sichert Zugriffe ab.

Der Lebenszyklus einer Queue



- Create: OS reserviert RAM für N Elemente der Größe X Bytes
- Put (Schreiben): Kopie der Daten hinten in die Queue
- Get (Lesen): vorderstes Element herausnehmen (wird entfernt!)
- Delete: RAM freigeben (auf MCUs oft ungenutzt — Queues leben ewig)

Notizen

Create einmal, dann Put/Get im Kreis. Get entfernt das Element — Platz wird frei. Delete gibt es, wird auf Mikrocontrollern kaum genutzt.

Speichermanagement einer Queue

- Vor der Nutzung dem OS zwei Dinge mitteilen:
 - Länge: maximal wie viele Elemente? (z. B. 10)
 - Elementgröße: Bytes pro Element? (z. B. `sizeof(uint32_t) = 4`)
- RAM-Bedarf: FreeRTOS reserviert sofort $10 \times 4 = 40$ Bytes aus dem Heap
 - plus OS-Overhead für die Queue-Verwaltung
- Dieser RAM ist reserviert, auch wenn die Queue leer ist

Notizen

Länge und Elementgröße festlegen. $10 \times 4 \text{ Bytes} = \text{mindestens } 40 \text{ Bytes Heap}$, auch bei leerer Queue.

Folie

12 / 45

Die Magie: Blockieren bei Leerlauf

- Was passiert, wenn der Consumer `Get ( )` aufruft und die Queue leer ist?
- Bare-Metal: leere Variable lesen oder in `while` festsitzen
- RTOS: Consumer sofort in den Blocked-Zustand
- Task verbraucht 0% CPU, bis der Producer ein `Put ( )` ausführt
- Sobald ein Wert eintrifft: Consumer wird (bei ausreichender Prio) sofort aufgeweckt

Notizen

Beste Task-Zustand aus VL3: Blocked. Timeout `osWaitForever` → leere Queue schickt Consumer schlafen (0% CPU). Put weckt sofort auf — kein Polling.

Folie

13 / 45

Die Magie: Blockieren bei Überlast

- Was passiert, wenn der Producer `Put ( )` aufruft und die Queue voll ist?
- Option A (Timeout = 0): Fehler `osErrorResource`, Wert verwerfen (Datenverlust)
- Option B (Timeout > 0): Producer geht in den Blocked-Zustand
 - Wartet, bis der Consumer Platz schafft
- Eleganter Mechanismus zur Drosselung (Throttling)

Notizen

Volle Queue: Timeout 0 = sofort Fehler/Datenverlust. Timeout > 0 = Producer schläft, bis Consumer liest. OS drosselt Überlast von allein.

Zusammenfassung: ITC-Vorteile

- Datentransfer: Werte sicher von Task A nach Task B
- Synchronisation: automatisches Blockieren und Wecken (keine leeren `while`-Schleifen)
- Pufferung: Geschwindigkeitsunterschiede (Bursts) ausgleichen
- Thread-Safe: keine eigenen Mutexe nötig — das OS regelt den Zugriff

Notizen

Queues: Transfer, Synchronisation, Pufferung und Thread-Safety in einem Mechanismus — Schweizer Taschenmesser der RTOS-Entwicklung.

CMSIS-API: Queue anlegen

- CMSIS-RTOS v2 Abstraktion:

```
// Globale Variable (Handle) für die Queue
osMessageQueueId_t sensorQueue;

void System_Init() {
    // Parameter 1: Anzahl der Elemente (z. B. 10)
    // Parameter 2: Größe eines Elements (z. B. sizeof(uint32_t))
    // Parameter 3: Optionale Attribute (meist NULL)
    sensorQueue = osMessageQueueNew(10, sizeof(uint32_t), NULL);

    if (sensorQueue == NULL) {
        // Fehler! Heap ist voll, kein RAM mehr.
    }
}
```

Notizen

Handle global, `osMessageQueueNew` vor Scheduler-Start. `sizeof` für plattformunabhängige Elementgröße. Rückgabe `NULL` prüfen: Heap voll.

CMSIS-API: Daten schreiben (Put)

- Der Producer legt Daten in die Queue:

```
void Task_ADC_Producer() {
    uint32_t value = 0;
    for(;;) {
        value = Read_ADC_Hardware();

        // Handle, Pointer auf Daten, Prio (immer 0), Timeout
        osStatus_t status = osMessageQueuePut(
            sensorQueue,
            &value,
            0,
            0 // Timeout = 0 (nicht warten!)
        );

        if (status != osOK) {
            // Queue ist voll, Wert verworfen!
        }
        osDelay(10); // Ticks; bei 1000 Hz = 10 ms
    }
}
```

Notizen

Adresse der Variable (&value): FreeRTOS kopiert die Bytes. Prio-Parameter historisch, in FreeRTOS immer 0. Timeout 0: bei voller Queue sofort Fehler, Wert verloren.

CMSIS-API: Daten lesen (Get)

- Der Consumer wartet auf Daten:

```

void Task_Filter_Consumer() {
    uint32_t received_value = 0;
    for(;;) {
        // Warte ewig (osWaitForever) auf neue Daten!
        osMessageQueueGet(
            sensorQueue,
            &received_value,
            NULL, // Prio-Pointer ignorieren
            osWaitForever
        );

        // Hier: ganz sicher ein gültiger Wert
        Filter_Data(received_value);
    }
}

```

Notizen

osWaitForever: leere Queue → Blocked an dieser Zeile. Erreicht der Code Filter_Data, liegt ein gültiges Paket vor — sauberer, deterministischer Flow.

Folie

18/45

Unter der Haube: Copy by Value

- FreeRTOS-Queues nutzen standardmäßig Copy by Value
- Put: OS kopiert den Wert byteweise in den eigenen Queue-RAM
- Get: OS kopiert den Wert zurück in die lokale Variable des Consumers
- Vorteil: Original-Variable des Producers ist danach völlig entkoppelt



Notizen

Zwei physikalische Kopien. Nach Put darf die Producer-Variable überschrieben, gelöscht oder der Scope verlassen werden — der Wert liegt sicher in der Queue.

Der Feind der Queue: Große Daten!

- Copy by Value ist ideal für kleine Daten (z. B. `uint32_t`, 4 Bytes)
- Große Daten? Kamerabild 320×240 Pixel, Graustufen (`uint8_t`) = 76 800 Bytes
- Probleme:
 - Queue wäre gigantisch (RAM-Verschwendung)
 - Zweimaliges Kopieren von ≈ 76 KB blockiert die CPU
 - `memcpy` zerstört Latenzgarantien
- Für große Daten ist Copy by Value streng verboten

Notizen

76 KB per Value: Heap reicht oft nicht; doppeltes Kopieren vernichtet Echtzeit. CPU nur noch Speicherschieben.

Die Lösung: Copy by Reference (Pointer)

- Statt 76 KB: nur die Speicheradresse (Pointer) in die Queue
- Pointer auf Cortex-M4: immer exakt 4 Bytes

```
// Queue für POINTER (Elementgröße = 4 Bytes!)
ptrQueue = osMessageQueueNew(10, sizeof(uint8_t*), NULL);

void Task_Camera() {
    uint8_t* image_buffer = Get_Image_From_Hardware();

    // Nicht das Bild, sondern die Adresse (4 Bytes)
    osMessageQueuePut(ptrQueue, &image_buffer, 0, 0);
}
```

- Consumer liest 4 Bytes, hat die Adresse und liest das Bild direkt aus dem RAM

Notizen

Bild bleibt im RAM. Queue speichert nur Pointer. Consumer folgt der Adresse — wenige Zyklen statt Millionen.

Gefahren von Copy by Reference

- Lebensgefahr: Producer darf `image_buffer` nicht mehr überschreiben, sobald der Pointer in der Queue liegt
- Sonst: Race Condition — Consumer liest korrupte Pixel
- Lösung: Buffer-Ping-Pong (Double Buffering)
 - Zwei getrennte Arrays
 - Producer beschreibt Array 1, sendet Pointer 1
 - Während Consumer Array 1 liest, beschreibt Producer Array 2

Notizen

Ohne schützende Kopie: Überschreiben vor Fertigstellung des Consumers = Race Condition. Double Buffering / Ping-Pong: zwei Puffer abwechselnd.

Der Spezialfall: Die Mailbox

- Oft verwechselt: Unterschied Queue vs. Mailbox?
- CMSIS/FreeRTOS: keine separate Mailbox-API — Spezialfall der Queue
- Mailbox = Queue der Länge 1
- Speichert exakt ein einziges Element
- Wichtigster Verhaltensunterschied: Überschreiben (Overwrite) ist ausdrücklich erwünscht

Notizen

Mailbox = Queue Länge 1. Semantik: bei Statusdaten ist absichtliches Überschreiben oft das Feature, nicht der Bug.

Wann Queue, wann Mailbox?

Queue (Events & Ströme)

- Tastendrücke (A, B, C abarbeiten)
- Audio-Daten
- Philosophie: kein Element darf verloren gehen
- Verhalten: blockiert, wenn voll

Mailbox (Zustand / Status)

- Aktuelle Temperatur, GPS-Position
- Philosophie: nur der neueste Wert zählt
- Verhalten: überschreibt alten Wert, blockiert nicht beim Schreiben
- FreeRTOS-Sonderfunktion: `xQueueOverwrite()`
- **Achtung:** CMSIS-RTOS v2 hat kein Overwrite/Pek — dafür native FreeRTOS-API nötig

Notizen

Queue: Events/Ströme ohne Verlust. Mailbox: Zustand (Kühlwasser) — Historie egal, nur aktueller Wert. Native FreeRTOS: `xQueueOverwrite`. CMSIS bietet das nicht — im Labor bei CMSIS-only also Queue Länge 1 + Put mit Timeout 0 als Näherung.

Das Broadcast-Konzept (Mailbox)

- Queue-Get entnimmt das Element — danach ist die Queue leer
- Status-Mailboxen: oft ein Producer, viele Consumer (Broadcast)
 - Sensor schreibt Drehzahl
 - Motor-, Display- und Logging-Task wollen lesen
- Peek (Spicken): Wert lesen, ohne ihn zu entfernen
 - `osMessageQueueGet()` entfernt
 - Native FreeRTOS: `xQueuePeek()`

Notizen

Bei Broadcast darf Get den Wert nicht vernichten. Peek kopiert und lässt den Wert in der Mailbox — sauberes State-Management.

Zusammenfassung Halbzeit

- Producer-Consumer: entkoppelt langsame Verarbeitung von schnellen Rohdaten
- Queues: Puffer im RAM, vom OS verwaltet
- Volle Queue blockiert beim Schreiben, leere Queue beim Lesen (0% CPU)
- Value vs. Reference: kleine Werte kopieren; große Daten (Bilder) als Pointer + Ping-Pong
- Mailbox: 1er-Queue für Zustände, die alte Daten überschreibt

Notizen

Erste Hälfte: Entkopplung durch Queues, Value vs. Reference, Mailboxen für aktuellen Sensorwert. Fragen zum Datenaustausch? Nach der Pause: Anbindung an echte Hardware-Interrupts.

Block 4: Interrupts im RTOS

- Wir wissen jetzt, wie Tasks untereinander reden
- Aber wie kommuniziert die Hardware (Außenwelt) mit unseren Tasks?
- Die Herausforderung:
 - Ein Hardware-Interrupt (ISR) reißt die CPU aus dem RTOS-Kontext heraus
 - Die ISR ist mächtiger als der Scheduler
 - Wie bringen wir Hardware-Events und Software-Scheduling sicher zusammen?

Notizen

Willkommen zurück nach der Pause! Bisher: Tasks, Queues, Mutexe unter Kontrolle des Schedulers. Ein MCU ohne Außenwelt ist wertlos. Tasten, Sensoren und Netzwerke laufen über Hardware-Interrupts (ISRs). ISRs kennen FreeRTOS nicht: sie grätschen in laufende Berechnungen, ignorieren Task-Prioritäten und können sogar den SysTick blockieren. Wie wir beide Welten verheiraten, ohne Echtzeit-Timing zu sprengen, ist das Thema dieses Blocks.

Rückblick: Die goldene Regel der ISR

- Erinnern Sie sich an Termin 1?
- Eine ISR muss extrem kurz sein!
- Warum ist das im RTOS noch wichtiger?
 - Solange die ISR läuft, ist die CPU komplett blockiert
 - Bei höherer HW-Priorität als SysTick: der RTOS-Herzschlag kann nicht feuern
 - Das Betriebssystem friert in der Zeit ein
 - Alle Deadlines aller Tasks verschieben sich nach hinten

Notizen

VL1-Regel: ISR kurz → kein `printf`, keine Zählschleife, kein Warten. Im RTOS überlebenswichtig: Timing über SysTick. Lange UART-ISR mit höherer HW-Priorität als SysTick → SysTick wird verzögert, OS verpasst Herzschläge, Zeit friert ein. Nach 10 ms ISR denkt das OS, es sei keine Millisekunde vergangen → Deadlines gescheitert.

Das Architektur-Muster: Deferred Interrupt Processing

- **Vertiefung VL4:** Semaphore-Signalisierung ISR → Task kennen Sie schon — hier der Architektur-Rahmen
- Wie lesen wir komplexe Sensordaten (z. B. 10 KB Bilddaten per SPI) ein, wenn die ISR kurz sein muss?
- Lösung der Industrie: Deferred Interrupt Processing (verzögerte Interrupt-Verarbeitung)
- Arbeit in zwei Hälften:
 1. Top-Half (die ISR): reagiert blitzschnell auf die Hardware
 2. Bottom-Half (die Task): übernimmt die zeitaufwändige Datenverarbeitung

Notizen

Dilemma: SPI-Bild plus CRC dauert. Nicht in der ISR → wo dann? Deferred Interrupt Processing: Top-Half und Bottom-Half. Muster in jedem modernen OS, von FreeRTOS bis Linux-Kernel.

Die Top-Half (Die ISR)

- Läuft im echten Hardware-Interrupt-Kontext (z. B. `EXTI15_10_IRQHandler`)
- Unterbricht alles im System
- Aufgaben der Top-Half:
 1. Hardware-Flag (Interrupt) löschen, damit er nicht endlos feuert
 2. Kritische Rohdaten aus Hardwareregistern (z. B. UART RDR) in den RAM retten
 3. Das RTOS wecken (signalisieren)
 4. Sofort beenden

Notizen

Top-Half = echte HW-ISR. Quittieren, Byte/Daten retten, Signal an RTOS, sofort Return. Dauer: oft unter einer Mikrosekunde.

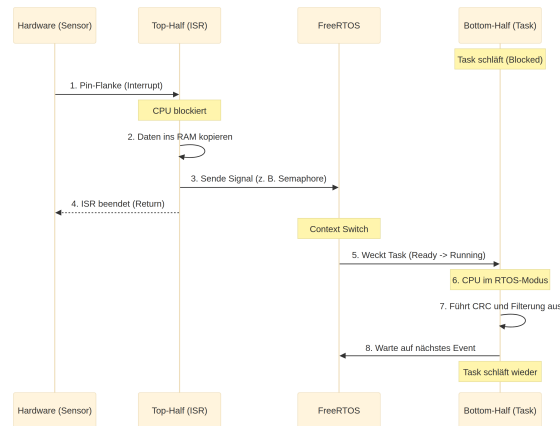
Die Bottom-Half (Die Task)

- Ist eine ganz normale C-Task in FreeRTOS
- Bekommt meist eine sehr hohe RTOS-Priorität
- Befindet sich zu 99% der Zeit im Zustand Blocked
- Aufgaben der Bottom-Half:
 1. Wartet auf das Signal der Top-Half (`osWaitForever`)
 2. Wacht auf
 3. Führt die aufwändige Logik durch (Filter, CRC, Float-Mathe, Display)
 4. Geht wieder schlafen

Notizen

Bottom-Half ist keine ISR, sondern eine FreeRTOS-Task mit `for ( ; ; )`. Hohe Prio, fast immer Blocked. Signal → Scheduler weckt Task. Im Task-Kontext läuft SysTick weiter → Millisekunden rechnen, Float, `printf` sind legal. Das System atmet.

Visualisierung: Top-Half und Bottom-Half



Notizen

Flow: Hardware alarmiert → Top-Half kopiert und signalisiert → ISR Return → OS weckt Bottom-Half → Task rechnet und blockiert erneut. Ping-Pong ISR/Task = Goldstandard der Embedded-Architektur.

Wie signalisiert die Top-Half?

- Die ISR muss der Bottom-Half eine Nachricht zukommen lassen
- Was eignet sich dafür perfekt? (Erinnerung an Termin 4)
- Der binäre Semaphor!
 - Bottom-Half: `osSemaphoreAcquire(..., osWaitForever)` und blockiert
 - Top-Half: `osSemaphoreRelease(...)`
 - Ergebnis: Task wacht auf → keine CPU-Zeit verschwendet
- Alternative für Daten: Queue (ISR: `osMessageQueuePut`)

Notizen

Signal = binärer Semaphor aus VL4: kein Ownership. Task: Acquire und schlafen. ISR: Release. Für Messwerte dasselbe Muster mit Queue: Task Get, ISR Put.

Die große Hürde: OS-Aufrufe in der ISR

- Massives Problem: OS-Funktionen (Put, Release) in einer Hardware-ISR
- OS-Funktionen können blockierend sein
- Was, wenn die Queue voll ist und die ISR `Put ( )` aufruft?
 - OS würde versuchen, die ISR in die Blocked-Liste zu schieben
 - Eine ISR kann man nicht schlafen legen
 - Die CPU würde mit einem fatalen `HardFault` abstürzen

Notizen

OS-APIs sind für Tasks geschrieben. Volle Queue → Task schläft. ISR hat keinen TCB, keine RTOS-Prio → Blockieren einer ISR ist physikalisch unmöglich und führt zum `HardFault`.

Native FreeRTOS-Lösung: Eigene ISR-Funktionen

- FreeRTOS besitzt eine doppelte API
- Für jede Funktion: Variante mit Suffix `...FromISR`
 - Task: `xSemaphoreGive (mySem)`
 - ISR: `xSemaphoreGiveFromISR (mySem, ...)`
- Unterschied: `FromISR`-Funktionen sind abgespeckt
 - Blockieren niemals (Queue: z. B. `errQUEUE_FULL`; Semaphore: `pdFALSE`)
 - Manipulieren Listen unterbrechungssicher

Notizen

Richard Barry: API verdoppelt. In nativer ISR niemals die normale API → immer `FromISR`. Kein Blockieren; Error-Code bei voller Queue. Ready-Listen ISR-sicher.

Die CMSIS-RTOS v2 Abstraktion

- CMSIS: keine doppelten Funktionen (`FromISR` existiert in der API nicht)
- `osSemaphoreRelease()` ist intelligent
 - Liest heimlich das ARM-Register IPSR (Interrupt Program Status Register)
 - Wenn `IPSR  $\neq$  0` (Handler Mode): wir sind in einer ISR → intern Umleitung auf `...FromISR`
 - Wenn `IPSR = 0` (Thread Mode): wir sind in einer Task → normaler Aufruf

Notizen

CMSIS-V2: immer `osSemaphoreRelease`. Wrapper prüft IPSR ($\neq 0$ = Interrupt). Automatisches Routing auf FreeRTOS-FromISR bzw. normale API. Riesiger Portabilitätsvorteil.

Die eine CMSIS-Regel für ISRs

- CMSIS nimmt uns fast alles ab → aber: eine ISR darf niemals warten!
- Goldene Regel: in einer ISR (Top-Half) muss `timeout` immer 0 sein
- `osWaitForever` in einer ISR → Aufruf schlägt fehl (`osErrorParameter`)

Notizen

Physik bleibt: ISR darf nicht warten. Timeout `osWaitForever` oder > 0 in ISR → CMSIS bricht mit `osErrorParameter` ab (kein Absturz). Merke: in der ISR ist der Timeout ausnahmslos 0.

Der Context-Switch aus der ISR (Yielding)

- Szenario:
 - Task A (Prio Low) läuft
 - ISR unterbricht Task A
 - ISR ruft `Release()` und weckt Task B (Prio High, Bottom-Half)
 - ISR beendet sich
- Was sollte passieren? Task B (High) sollte sofort rechnen
- Aber: OS schaltet standardmäßig erst beim nächsten SysTick (bis zu 1 ms)
- Folge: bis zu 1 ms Reaktionslatenz verschenkt

Notizen

Nach ISR-Return springt die CPU oft zurück in die Low-Prio-Task. High-Prio ist Ready, aber Scheduler wartet auf SysTick. Bis zu 1 ms Latenz verschenkt.

Folie

38 / 45

Wie löst FreeRTOS das Yielding?

- Die ISR muss dem OS sagen: wichtige Task geweckt → wechsele sofort nach ISR-Ende
- Native FreeRTOS über Parameter:

```
BaseType_t xHigherPriorityTaskWoken = pdFALSE;

// true, wenn Task B wichtiger ist als Task A
xSemaphoreGiveFromISR(mySem, &xHigherPriorityTaskWoken);

// Fordert sofortigen Context-Switch (PendSV) an
portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
```

- Dadurch triggert die ISR den PendSV-Interrupt manuell

Notizen

Native API: Pointer `xHigherPriorityTaskWoken`, danach `portYIELD_FROM_ISR`. Makro setzt PendSV-Bit. Nach ISR-Ende: PendSV-Handler statt Rückkehr zur alten Task.

Folie

39 / 45

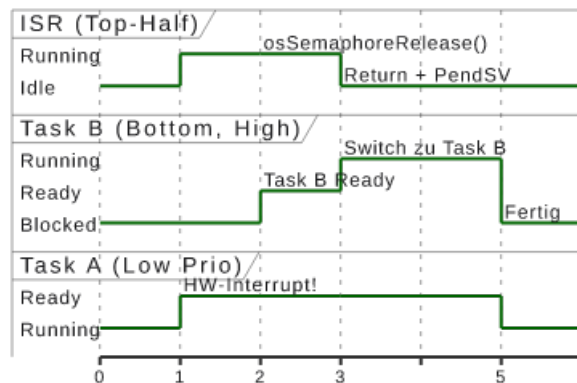
CMSIS ist unser Freund!

- Müssen wir `portYIELD_FROM_ISR` in CMSIS schreiben?
- Nein!
- Wrapper `osSemaphoreRelease()` macht Yielding intern automatisch
- Höhere Prio der geweckten Task → CMSIS triggert PendSV von allein
- Sie schreiben nur: `osSemaphoreRelease(mySem);`

Notizen

CMSIS-V2: FromISR-Umleitung und Yield-Handling automatisch. Eine saubere Zeile in der HW-ISR: `osSemaphoreRelease`.

Visualisierung: Präemption aus der ISR



- Latenz zwischen Hardware-Event ($t = 1$) und Start der Bottom-Half ($t = 3$) ist minimal

Notizen

$t = 1$: HW-Interrupt. $t = 2$: Release stellt Task B Ready (noch nicht Running). $t = 3$: ISR-Return + PendSV → CPU an Task B. Task B fertig, bevor Task A es merkt → präemptives Interrupt-Handling.

Laborvorbereitung: Deferred Interrupts

- Ziel: Hardware-Taster via EXTI-Interrupt und Semaphor an FreeRTOS-Task anbinden
- Hardware: blauer User-Button (B1) auf dem STM32H755 Nucleo, meist Pin PC13
- Setup in CubeMX:
 - Pin PC13 als `GPIO_EXTI13` (External Interrupt)
 - NVIC: Haken bei `EXTI line[15:10] interrupts` (sonst feuert die ISR nicht)
 - FreeRTOS: binären Semaphor `ButtonSem` anlegen
 - Task `ButtonTask` (Bottom-Half) mit Priorität High

Notizen

Labor: blauer Button an PC13. CubeMX: EXTI13, NVIC-Haken setzen! Binary Semaphore + ButtonTask mit hoher Priorität.

Schritt 1: Die Top-Half (ISR) in C

- CubeIDE generiert die ISR; wir nutzen den sicheren HAL-Callback
- In `main.c` den leeren `HAL_GPIO_EXTI_Callback` füllen:

```
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == USER_Btn_Pin) { // War es unser Taster?
        // 1. Wecke die Bottom-Half auf
        osSemaphoreRelease(ButtonSemHandle);

        // 2. ISR sofort beenden
        // (CMSIS fordert intern PendSV an)
    }
}
/* USER CODE END 4 */
```

Notizen

HAL fängt den Assembler-Interrupt ab, löscht das Flag, ruft den Callback. Nur prüfen und `osSemaphoreRelease` → keine `printf`, keine Berechnung. Perfekte kurze Top-Half.

Schritt 2: Die Bottom-Half (Task) in C

- In der Endlosschleife der Task auf das Signal warten:

```

void StartButtonTask(void *argument)
{
    for (;;) {
        // 1. Schlafe (0% CPU), bis ISR signalisiert
        osSemaphoreAcquire(ButtonSemHandle, osWaitForever);

        // 2. WACH - hier die aufwändige Arbeit
        HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
        printf("BUTTON PRESSED!\n");

        // NUR DEMO: busy-wait, Task bleibt Running!
        HAL_Delay(100);
        // Produktion: osDelay(100); // Ticks; bei 1000 Hz = 100
    }
}

```

Notizen

Acquire mit `osWaitForever` → Blocked. Nach Knopfdruck: LED, `printf`, Demo-Last. `HAL_Delay` busy-waitet und hält die Task im Running-Zustand → nur zur Demo. In Produktion `osDelay` (Ticks!), damit die Task Blocked geht und andere Tasks rechnen können. In der ISR hätte jedes Delay den Chip gekillt; im Task-Kontext überlebt das OS.

Folie

44 / 45

Was werden wir im Labor beobachten?

- Button drücken → LED und Console verzögerungsfrei (für das Auge)
- Beweis der RTOS-Architektur:
 - Trotz `printf` und Delay stürzen parallel laufende Tasks (z. B. blinkende LED) nicht ab
 - Eine noch wichtigere Task könnte unser `printf` präemptieren
- Hardware-Interrupts und Software-Scheduling sind sicher vereint

Notizen

Keine Polling-Latenz, ISR-Regeln eingehalten, Scheduler bleibt Herr der Lage. Demo-Delay auf Prio High; Realtime-Task mit höherer Prio könnte präemptieren. Finale der Woche: Hardware-Welt und FreeRTOS vereint.

Zusammenfassung des Tages

- Queues: transportieren Daten (Values oder Pointer) threadsicher zwischen Tasks; blockieren bei Über-/Unterlast
- Mailboxes: Zustandskommunikation (Overwrite erlaubt)
- ISRs müssen extrem kurz sein, damit der RTOS-SysTick nicht blockiert
- Deferred Interrupt Processing: Top-Half-ISR löst nur ein Event (Semaphor) aus; Bottom-Half-Task erledigt die Arbeit
- CMSIS-RTOS v2: FromISR-Logik und Context-Yielding automatisch

Notizen

Heute: Queues (Value/Pointer), Mailboxes, Top-/Bottom-Half über binären Semaphor in CMSIS. Ins Labor → EXTI-Interrupt ausprobieren. Viel Erfolg!